

WaBot AI

Technical Specification Document v2.0

Baileys WA Engine

OpenRouter AI Integration

Multi-Tenant Isolation

Interactive Onboarding

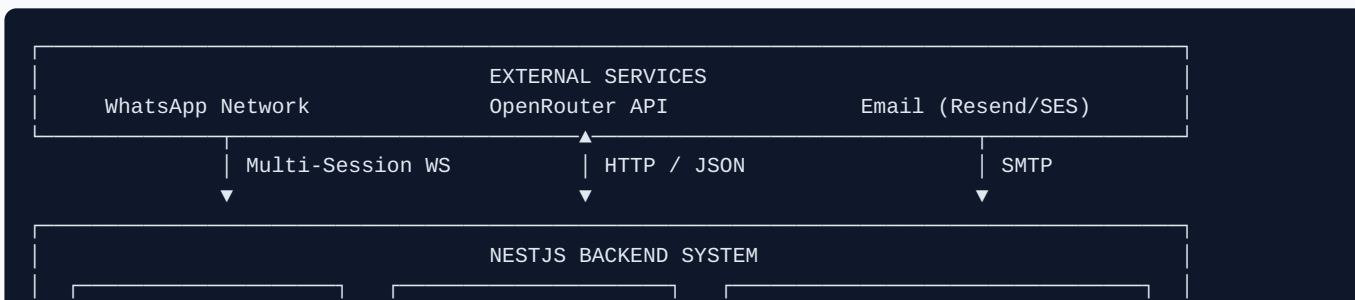
Nama Produk	WaBot AI Platform
Versi Dokumen	v2.0 (Agustus 2026 Revision)
Backend Tech Stack	NestJS 10 + TypeScript 5 + Prisma ORM 5
Frontend Tech Stack	Next.js 14 (App Router) + Tailwind CSS + shadcn/ui
Data & Queue Store	PostgreSQL 16 + Redis 7 + BullMQ Queue
WA Gateway	Baileys JS Engine (Multi-Session Websocket Client & QR Code Sync)
AI Engine Provider	OpenRouter API (Unified Router for Claude, GPT-4o, Llama 3, DeepSeek)
Arsitektur Tenant	Multi-Tenant (Row-Level Security / Business Account Isolation & Organization Onboarding)

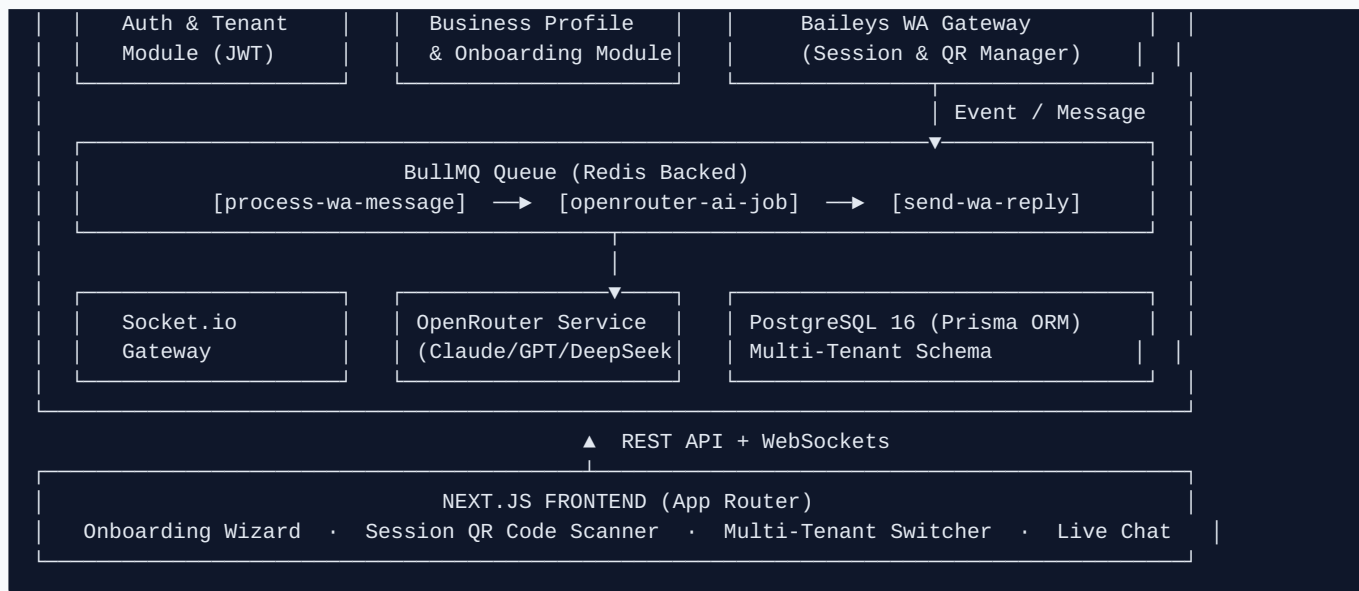
1. Arsitektur Sistem & Perubahan Utama (v2.0)

Dokumen spesifikasi teknis ini merupakan pembaruan komprehensif dari v1.0. Terdapat empat pilar arsitektur utama yang diperbarui untuk meningkatkan efisiensi operasional, menurunkan biaya API, dan mendukung fleksibilitas multi-tenant secara independen:

- Migrasi WA Engine ke Baileys:** Menggantikan Meta WhatsApp Cloud API dengan `@whiskeysockets/baileys`. Memungkinkan pengiriman pesan langsung via WebSocket session tanpa biaya per pesan Meta API dan memfasilitasi scanning QR code mandiri oleh tenant.
- Migrasi AI Router ke OpenRouter:** Menggantikan integrasi langsung Anthropic SDK dengan OpenRouter API. Memungkinkan tenant memilih/mengganti LLM secara fleksibel (Claude 3.5 Sonnet, GPT-4o, DeepSeek V3, Llama 3) serta mempermudah pengawasan biaya token.
- Strict Multi-Tenant Architecture:** Pemisahan data dan konfigurasi bisnis yang ketat berorientasi pada `BusinessAccount` dan `TenantMember`. Mendukung role-based access control (Owner, Admin, Agent).
- User & Tenant Onboarding Flow:** Wizard interaktif bertahap untuk akun baru (Registrasi → Profile Setup → QR Scan Baileys → AI Prompt/LLM Config → Product Ingestion).

1.1 High-Level Architecture Diagram





1.2 Message Flow — Inbound Message (Baileys + OpenRouter)

1. Pesan masuk dari pelanggan ke WhatsApp HP yang terhubung.
2. `BaileysService` menangkap event `messages.upsert` untuk `sessionId` tenant tersebut.
3. Handler mengecek apakah sender terdaftar atau nomor bisnis sendiri (ignore self-messages).
4. Sistem menyimpan raw message ke DB dengan `businessAccountId` dan memicu Socket.io event ke Dashboard.
5. Job dipush ke BullMQ Queue: `process-wa-message`.
6. Worker mengambil profil bisnis, instruksi sistem, katalog produk, dan histori chat (10 pesan terakhir) milik tenant tersebut.
7. Worker mengirim prompt ke **OpenRouter API** menggunakan model yang dikonfigurasi tenant (default: `anthropic/claude-3.5-sonnet` atau `deepseek/deepseek-chat`).
8. OpenRouter mengembalikan balasan AI. Worker mengecek aturan eskalasi (misal keyword "human", "komplain").
9. Job dipush ke Queue: `send-wa-reply`.
10. `BaileysService` mengirim balasan ke pelanggan via active WebSocket connection.
11. Status message diupdate ke `REPLIED` dan data statistik diperbarui.

2. Struktur Folder Project

2.1 Backend — NestJS Structure

```

wabot-backend/
├── src/
│   ├── main.ts
│   ├── app.module.ts
│   └── modules/
│       ├── auth/                # Auth JWT, Refresh token, Tenant Guard
│       ├── tenant/              # Multi-tenant management & Memberships
│       ├── onboarding/          # Step-by-step onboarding wizard handler
│       ├── business/            # Profile, Operating hours, Escalation settings
│       ├── baileys/             # WhatsApp Baileys Engine
│       │   ├── baileys.module.ts
│       │   ├── baileys.service.ts # Session manager, QR code emitter
│       │   └── baileys.listener.ts # Event listener (messages.upsert, connection.update)
│       ├── openrouter/          # OpenRouter Integration
│       │   ├── openrouter.module.ts
│       │   └── openrouter.service.ts # OpenRouter API client & Model selector
  
```

```

├── chatbot/           # Context builder & prompt orchestrator
├── conversations/     # Chat histories & status management
├── products/          # Catalog management for AI context
├── analytics/         # Token usage & message stats
├── queue/             # BullMQ Processors
├── process-message.processor.ts
├── send-wa-reply.processor.ts
├── gateway/           # Realtime QR & Chat Gateways
├── chat.gateway.ts
├── prisma/
├── common/
├── decorators/        # @CurrentTenant(), @Roles()
├── guards/            # TenantAccessGuard, JwtAuthGuard

```

2.2 Frontend — Next.js Structure

```

wabot-frontend/
├── src/
├── app/
│   ├── (auth)/           # Login, Register
│   ├── (onboarding)/     # Interactive Setup Wizard
│   │   ├── step-1-profile/page.tsx
│   │   ├── step-2-wa-connect/page.tsx # QR Code Baileys scanner
│   │   ├── step-3-ai-setup/page.tsx  # System prompt & Model selection
│   │   └── step-4-catalog/page.tsx
│   └── (dashboard)/      # Main Dashboard (Tenant Isolated)
│       ├── [tenantId]/
│       │   ├── overview/
│       │   ├── conversations/
│       │   ├── wa-sessions/ # Manage Baileys connection status & reconnect
│       │   ├── ai-settings/ # OpenRouter config (Model choice, Temp, Prompt)
│       │   ├── products/
│       │   └── settings/
│       ├── components/
│       │   ├── onboarding/ # Step indicators & wizard controls
│       │   ├── baileys/    # QR Code display component
│       │   └── chat/       # Live conversation viewer
│       └── lib/
│           ├── openrouter-models.ts # Model metadata (Claude, GPT-4o, DeepSeek)
│           └── api.ts

```

3. Database Schema (Prisma Multi-Tenant & Baileys)

```

// prisma/schema.prisma

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

enum Role {
  OWNER
  ADMIN
  AGENT
}

enum SessionStatus {

```

```

DISCONNECTED
SCAN_QR_NEEDED
CONNECTING
CONNECTED
}

enum ConvStatus {
  AI_HANDLING
  NEEDS_HUMAN
  HUMAN_HANDLING
  RESOLVED
}

enum MessageRole {
  CUSTOMER
  AI
  HUMAN_AGENT
}

model User {
  id          String          @id @default(cuid())
  email       String          @unique
  password    String
  name        String
  refreshToken String?
  onboarded   Boolean         @default(false)
  createdAt   DateTime        @default(now())
  updatedAt   DateTime        @updatedAt
  memberships TenantMember[]
  subscriptions Subscription?
}

model BusinessAccount {
  id          String          @id @default(cuid())
  slug        String          @unique
  name        String
  description  String?        @db.Text
  category    String?
  operatingHours Json?
  escalationPhone String?
  createdAt   DateTime        @default(now())
  updatedAt   DateTime        @updatedAt

  members      TenantMember[]
  waSessions    WaSession[]
  conversations Conversation[]
  products      Product[]
  aiConfig      AiConfig?
}

model TenantMember {
  id          String          @id @default(cuid())
  userId      String
  businessAccountId String
  role        Role           @default(OWNER)
  createdAt   DateTime        @default(now())

  user          User          @relation(fields: [userId], references: [id], onDelete: Cascade)
  businessAccount BusinessAccount @relation(fields: [businessAccountId], references: [id], onDelete: Cascade)

  @@unique([userId, businessAccountId])
}

model WaSession {
  id          String          @id @default(cuid())

```

```

    businessAccountId String
    sessionId         String          @unique // Unik ID Baileys per tenant
    phoneNumber       String?
    displayName       String?
    status            SessionStatus    @default(DISCONNECTED)
    qrCode            String?          @db.Text
    authKeys          Json?            // Encrypted Baileys creds/session dump
    createdAt         DateTime         @default(now())
    updatedAt         DateTime         @updatedAt

    businessAccount   BusinessAccount @relation(fields: [businessAccountId], references: [id], onDelete:
Cascade)
    conversations     Conversation[]
}

model AiConfig {
    id                String           @id @default(cuid())
    businessAccountId String           @unique
    provider          String           @default("openrouter")
    model             String           @default("anthropic/claude-3.5-sonnet") // OpenRouter Model ID
    temperature       Float            @default(0.7)
    tone              String           @default("friendly")
    language          String           @default("id")
    customSystemPrompt String?         @db.Text
    escalationKeywords String[]
    maxHistoryMessages Int             @default(10)

    businessAccount   BusinessAccount @relation(fields: [businessAccountId], references: [id], onDelete:
Cascade)
}

model Conversation {
    id                String           @id @default(cuid())
    businessAccountId String
    waSessionId       String
    customerPhone     String
    customerName      String?
    status            ConvStatus       @default(AI_HANDLING)
    lastMessageAt     DateTime?
    createdAt         DateTime         @default(now())

    businessAccount   BusinessAccount @relation(fields: [businessAccountId], references: [id], onDelete:
Cascade)
    waSession         WaSession        @relation(fields: [waSessionId], references: [id])
    messages          Message[]
}

model Message {
    id                String           @id @default(cuid())
    conversationId    String
    role              MessageRole
    content           String           @db.Text
    mediaUrl          String?
    waMessageId       String?
    modelUsed         String?          // e.g. "anthropic/claude-3.5-sonnet"
    tokensUsed        Int?
    latencyMs         Int?
    createdAt         DateTime         @default(now())

    conversation      Conversation @relation(fields: [conversationId], references: [id], onDelete: Cascade)
}

model Product {
    id                String           @id @default(cuid())
    businessAccountId String
    name              String

```

```

description      String?          @db.Text
price            Decimal?
stock           Int?
imageUrl         String?
isActive        Boolean          @default(true)
createdAt        DateTime         @default(now())

businessAccount  BusinessAccount @relation(fields: [businessAccountId], references: [id], onDelete:
Cascade)
}

model Subscription {
  id              String          @id @default(cuid())
  userId          String          @unique
  plan            String          @default("STARTER")
  status          String          @default("ACTIVE")
  convUsed        Int             @default(0)
  convLimit       Int             @default(500)
  periodStart     DateTime
  periodEnd       DateTime

  user            User            @relation(fields: [userId], references: [id])
}

```

4. API Endpoints & Onboarding Workflow

4.1 Onboarding Flow Endpoints

Method	Endpoint	Deskripsi	Auth
POST	/api/onboarding/start	Inisialisasi tenant & profil bisnis pertama user	JWT
GET	/api/onboarding/status	Mengecek progress onboarding user (Step 1-4)	JWT
POST	/api/onboarding/wa-session	Request pembuatan Baileys QR code untuk tenant	JWT + TenantGuard
POST	/api/onboarding/ai-config	Menyimpan konfigurasi OpenRouter & prompt awal	JWT + TenantGuard
POST	/api/onboarding/complete	Menandai user <code>onboarded = true</code> dan redirect ke dashboard	JWT

4.2 Multi-Tenant WhatsApp & Baileys Session Management

Method	Endpoint	Deskripsi	Auth
GET	/api/tenant/:tenantId/wa/qrcode	Mendapatkan QR Code ter-generate via WebSocket/ HTTP	JWT + TenantGuard
POST	/api/tenant/:tenantId/wa/reconnect	Picu pendaftaran ulang session Baileys	JWT + TenantGuard
POST	/api/tenant/:tenantId/wa/disconnect	Logout session WA dan hapus auth keys dari Redis/DB	JWT + TenantGuard

4.3 OpenRouter AI Configuration Endpoints

Method	Endpoint	Deskripsi	Auth
GET	/api/tenant/:tenantId/ai-config	Get setting AI (OpenRouter model, temp, prompt)	JWT + TenantGuard
PUT	/api/tenant/:tenantId/ai-config	Update model OpenRouter (Claude, DeepSeek, GPT-4o)	JWT + TenantGuard
GET	/api/ai/models	Fetch daftar model aktif yang tersedia di OpenRouter	JWT

5. Implementasi Modul Utama

5.1 OpenRouter Service Integration (NestJS)

```
// src/modules/openrouter/openrouter.service.ts
import { Injectable, Logger } from '@nestjs/common';
import { ConfigService } from '@nestjs/config';
import axios from 'axios';

@Injectable()
export class OpenRouterService {
  private readonly logger = new Logger(OpenRouterService.name);
  private readonly apiKey: string;

  constructor(private configService: ConfigService) {
    this.apiKey = this.configService.get<string>('OPENROUTER_API_KEY');
  }

  async generateReply(params: {
    model: string;
    temperature: number;
    systemPrompt: string;
    history: { role: 'user' | 'assistant'; content: string }[];
    userMessage: string;
  }): Promise<{ reply: string; tokensUsed: number; latencyMs: number }> {
    const startTime = Date.now();
    const messages = [
      { role: 'system', content: params.systemPrompt },
      ...params.history,
      { role: 'user', content: params.userMessage },
    ];

    try {
      const response = await axios.post(
        'https://openrouter.ai/api/v1/chat/completions',
        {
          model: params.model || 'anthropic/claude-3.5-sonnet',
          temperature: params.temperature ?? 0.7,
          messages,
        },
        {
          headers: {
            Authorization: `Bearer ${this.apiKey}`,
            'HTTP-Referer': 'https://wabot.ai',
            'X-Title': 'WaBot AI Multi-Tenant',
            'Content-Type': 'application/json',
          },
        },
      );

      const latencyMs = Date.now() - startTime;
```

```

    const reply = response.data.choices[0]?.message?.content || '';
    const tokensUsed = response.data.usage?.total_tokens || 0;

    return { reply, tokensUsed, latencyMs };
  } catch (error) {
    this.logger.error('Error calling OpenRouter API:', error.response?.data || error.message);
    throw new Error('Gagal mendapatkan respon dari AI Router.');
  }
}
}
}

```

5.2 Baileys WA Engine Handler (NestJS)

```

// src/modules/baileys/baileys.service.ts
import { Injectable, OnModuleInit, Logger } from '@nestjs/common';
import makeWASocket, { disconnectReason, useMultiFileAuthState } from '@whiskeysockets/baileys';
import { PrismaService } from '../../prisma/prisma.service';
import { ChatGateway } from '../../gateway/chat.gateway';

@Injectable()
export class BaileysService implements OnModuleInit {
  private sessions = new Map<string, any>();
  private readonly logger = new Logger(BaileysService.name);

  constructor(
    private prisma: PrismaService,
    private chatGateway: ChatGateway,
  ) {}

  async onModuleInit() {
    // Restore all active session on startup
    const activeSessions = await this.prisma.waSession.findMany({
      where: { status: 'CONNECTED' },
    });
    for (const session of activeSessions) {
      this.initSession(session.businessAccountId, session.sessionId);
    }
  }

  async initSession(businessAccountId: string, sessionId: string) {
    const { state, saveCreds } = await useMultiFileAuthState(`./sessions/${sessionId}`);

    const sock = makeWASocket({
      auth: state,
      printQRInTerminal: false,
    });

    this.sessions.set(sessionId, sock);

    sock.ev.on('creds.update', saveCreds);

    sock.ev.on('connection.update', async (update) => {
      const { connection, lastDisconnect, qr } = update;

      if (qr) {
        await this.prisma.waSession.update({
          where: { sessionId },
          data: { qrCode: qr, status: 'SCAN_QR_NEEDED' },
        });
        this.chatGateway.emitQrCode(businessAccountId, qr);
      }

      if (connection === 'open') {
        await this.prisma.waSession.update({

```



```

        where: { sessionId },
        data: { status: 'CONNECTED', qrCode: null, phoneNumber: sock.user.id.split(':')[0] },
    });
    this.chatGateway.emitSessionStatus(businessAccountId, 'CONNECTED');
  }
});

sock.ev.on('messages.upsert', async (m) => {
  if (m.type === 'notify') {
    for (const msg of m.messages) {
      if (!msg.key.fromMe) {
        // Trigger BullMQ queue to process inbound WA message
        await this.handleInboundMessage(businessAccountId, sessionId, msg);
      }
    }
  }
});
}

async sendMessage(sessionId: string, toPhone: string, text: string) {
  const sock = this.sessions.get(sessionId);
  if (!sock) throw new Error('WhatsApp session tidak terhubung');
  const jid = `${toPhone.replace(/^[0-9]/g, '')}@s.whatsapp.net`;
  await sock.sendMessage(jid, { text });
}

private async handleInboundMessage(businessAccountId: string, sessionId: string, msg: any) {
  // Pipeline parsing & queue dispatch
}
}

```

6. User Onboarding Setup (Frontend Next.js)

User onboarding didesain menggunakan aliran wizard 4 langkah (Step-by-Step UI) yang memandu pemilik bisnis dari pendaftaran awal hingga AI siap membalas pesan secara otomatis:

Onboarding Flow Steps:

- Step 1: Profil Bisnis:** Pengisian nama bisnis, kategori, jam kerja, dan nomor WhatsApp eskalasi owner.
- Step 2: WhatsApp Baileys Pairing:** Sistem menampilkan QR Code Baileys secara realtime via Socket.io. User melakukan scan QR melalui aplikasi WhatsApp di HP.
- Step 3: AI Engine & OpenRouter Config:** Memilih model AI (Claude 3.5 Sonnet / GPT-4o / DeepSeek V3), mengisi nada bicara (*tone*), dan menyusun kustom *system prompt*.
- Step 4: Katalog Produk Context:** Mengunggah/memasukkan data produk dasar yang digunakan AI sebagai pengetahuan dalam membalas pertanyaan calon pembeli.

7. Environment Variables (v2.0)

7.1 Backend (.env)

```

# Application
NODE_ENV=production
PORT=3001

# Multi-Tenant & Database
DATABASE_URL="postgres://wabot_user:password@localhost:5432/wabot_db?schema=public"

```

```
# Cache & BullMQ Queue
REDIS_HOST=localhost
REDIS_PORT=6379
REDIS_PASSWORD=secret_redis

# Authentication
JWT_SECRET=super_secret_jwt_access_key
JWT_EXPIRES_IN=15m
JWT_REFRESH_SECRET=super_secret_jwt_refresh_key
JWT_REFRESH_EXPIRES_IN=7d

# OpenRouter Unified API
OPENROUTER_API_KEY=sk-or-v1-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

# Baileys WhatsApp Configurations
BAILEYS_SESSION_DIR=./sessions

# File Upload / Storage
STORAGE_LOCAL_PATH=./uploads
```

8. Multi-Tenant Security & Isolation Strategy

- **Row-Level Tenant Filtering:** Setiap request yang berhasil lolos dari `JwtAuthGuard` dilewatkan ke `TenantAccessGuard`. Guard membaca header `x-tenant-id` atau URL param `tenantId`, mengonfirmasi keanggotaan user di tabel `TenantMember`.
- **Prisma Middleware Isolation:** Semua query database wajib mengeksekusi penyaringan `where: { businessAccountId }` untuk mencegah kebocoran data antar tenant.
- **Baileys Session Sandboxing:** Kredensial koneksi WhatsApp disimpan di folder terisolasi `/sessions/{sessionId}` dan status dikunci khusus untuk `businessAccountId` terkait.